

AWS ML Engineer Associate

Section 8 Amazon Bedrock

Building GenAI Apps with Foundation Models on AWS

Topics Covered

- ✓ Amazon Bedrock Overview & API Endpoints
- ✓ IAM Permissions & Model Access
- ✓ Fine-Tuning & Custom Models
- ✓ Continued Pre-Training
- ✓ Retrieval Augmented Generation (RAG)
- ✓ Embeddings & Vector Databases
- ✓ Bedrock Knowledge Bases
- ✓ Amazon Bedrock Guardrails
- ✓ LLM Agents in Bedrock
- ✓ Deploying Bedrock Agents
- ✓ More Bedrock Features
- ✓ Master Cheat Sheet

Sundog Education — MLA-C01 Exam Prep

TABLE OF CONTENTS

Section 8 — Amazon Bedrock

1. Amazon Bedrock Overview	3
2. The Bedrock API Endpoints	4
3. IAM Permissions & Model Access	5
4. Fine-Tuning in Bedrock (Custom Models)	6
5. Continued Pre-Training	7
6. Retrieval Augmented Generation (RAG)	8
7. Embeddings & Vector Databases	10
8. Bedrock Knowledge Bases	11
9. Amazon Bedrock Guardrails	12
10. LLM Agents in Bedrock	13
11. Deploying Bedrock Agents	15
12. More Bedrock Features	16
13. Master Cheat Sheet	17

1. AMAZON BEDROCK OVERVIEW

Foundation Models as a Service

Amazon Bedrock is AWS's fully serverless, managed API for accessing and building applications on top of **Foundation Models (FMs)** — large, pre-trained AI models capable of text, image, and embedding tasks. You do not manage any infrastructure; AWS handles capacity, scaling, and availability automatically.

What Can Bedrock Do?

Core Capabilities

- **Invoke any foundation model** — chat, text completion, or image generation via a unified API.
- **Access pre-built models** from Anthropic (Claude), AI21 Labs, Cohere, Meta (Llama), Mistral AI, Stability AI, and Amazon (Titan).
- **Fine-tune models** on your own data to specialise them for your use case.
- **Import custom models** you built elsewhere (from S3 or SageMaker).
- **RAG support** — connect a Knowledge Base for Retrieval-Augmented Generation.

Additional Features

- **LLM Agent support** — build intelligent agents that can call tools (Lambda functions).
- **Guardrails** — content filtering for prompts and responses.
- **Serverless** — pay only per token/request; no cluster management.
- **SageMaker Canvas integration** — use Bedrock models directly from Canvas.
- **Third-party billing** — third-party model costs pass through AWS billing.

■ Exam Tip — Bedrock Is Always Serverless

A common distractor on the exam is suggesting you need to provision instances for Bedrock. **Bedrock is 100% serverless** — no EC2, no cluster management. If a question asks which service lets you use FM APIs without managing infrastructure, the answer is Bedrock.

Supported Foundation Model Providers

Provider	Models	Strengths
AI21 Labs	Jurassic series	Text completion, summarisation
Amazon	Titan (Text, Embeddings, Image)	Text, embeddings, image generation — AWS-native
Anthropic	Claude (Sonnet, Haiku, Opus)	Text & vision — strong reasoning
Cohere	Command, Command R+	Text — enterprise NLP
Meta	Llama 3 series	Open-weight text generation
Mistral AI	Mistral, Mixtral	Fast, efficient text generation
Stability AI	Stable Diffusion	Image generation from text

■ Key Note — Third-Party Pricing

When you use a third-party FM (e.g., Claude, Llama, Jurassic), you agree to that provider's EULA and are billed at their rates — but conveniently **through your AWS bill**. Amazon Titan models are AWS-native and approve access immediately. Third-party models may require additional information and a few minutes to approve.

2. THE BEDROCK API ENDPOINTS

Four Key Namespaces to Know

Bedrock is split into **four API namespaces**, each with a distinct purpose. Understanding the split between management and inference, and between base models and agents, is important for both exam questions and real-world usage.

Endpoint	Purpose	Key Operations
bedrock	Control plane — manage, deploy, and train models.	CreateModelCustomizationJob, GetFoundationModel, ListFoundationModels, CreateProvisionedModelThroughput
bedrock-runtime	Inference plane — execute prompts and generate embeddings against models.	Converse, ConverseStream, InvokeModel, InvokeModelWithResponseStream
bedrock-agent	Control plane — manage, deploy, and train LLM agents and knowledge bases.	CreateAgent, CreateKnowledgeBase, AssociateAgentKnowledgeBase
bedrock-agent-runtime	Inference plane — perform inference against agents and knowledge bases.	InvokeAgent, Retrieve, RetrieveAndGenerate

Key API Calls — Explained

Converse / ConverseStream

The recommended way to invoke chat-based models. Abstracts model-specific request formats. ConverseStream returns a streaming response token by token.

InvokeModel / InvokeModelWithResponseStream

Lower-level invocation that accepts raw model-specific JSON. Used when you need full control over the request body or when Converse does not support a particular model or modality.

Retrieve

Performs a semantic search against a Bedrock Knowledge Base. Returns relevant document chunks without generating a text response.

RetrieveAndGenerate

Combines retrieval from a Knowledge Base with text generation. Bedrock automatically builds the augmented prompt and returns a grounded answer.

InvokeAgent

Sends a user request to a deployed Bedrock Agent. The agent plans, calls action groups (Lambda), and queries knowledge bases to generate a final response.

■ Exam Tip — Runtime vs Control Plane

"-runtime" endpoints = inference (calling/running models). Endpoints without "-runtime" = management (creating, configuring, training). If an exam question asks which endpoint to use to "execute a prompt", it's always a runtime endpoint.

3. IAM PERMISSIONS & MODEL ACCESS

Security Before You Can Use Bedrock

IAM Requirements

Bedrock enforces IAM authentication strictly. You **cannot use the root account** to call Bedrock APIs — you must use an IAM user, role, or service account. This follows the AWS security best practice of least privilege.

IAM Policy	Type	What It Allows
AmazonBedrockFullAccess	AWS Managed	Full access: invoke models, manage knowledge bases, create agents, fine-tune models.
AmazonBedrockReadOnly	AWS Managed	Read-only: list and describe models and resources. Cannot invoke or manage.
AmazonSageMakerCanvasBedrockAccess	AWS Managed	Allows SageMaker Canvas to call Bedrock models on behalf of users.
Custom inline policies	Customer Managed	Grant granular access, e.g., only InvokeModel on specific model ARNs.

Model Access: Request Before You Use

Even with correct IAM permissions, you **must first request access** to each foundation model before you can use it. This is a one-time step per account per region.

Model Access Process

- **Amazon Titan models:** Access granted immediately upon request. No extra info needed.
- **Third-party models** (e.g., Claude, Llama): May require you to agree to the provider's EULA and submit additional information.
- **Approval time:** Usually just a few minutes for third-party models.
- **Billing:** Once approved, usage is billed at the third party's rates through your AWS bill.

Things to Know

- **Access status** shows in the Bedrock console: "Access granted" or "Available to request".
- **Per-region:** Model access must be requested in each AWS region where you want to use it.
- **Pricing:** Always check aws.amazon.com/bedrock/pricing — each model has different per-token rates.
- **Playground:** After access is granted, you can test models in the Bedrock Playground (Chat, Text, Image).

■ Exam Tip — Model Access Gate

A very common gotcha: having the right IAM permissions does NOT mean you can immediately call a model. **Model access must be requested separately** in the Bedrock console. Exam scenarios may describe a "ModelNotAccessible" error — the fix is to request model access, not to change IAM.

■ Pro Tip — Bedrock Playground

The Bedrock Playground (Chat / Text / Image environments) is useful for quick experimentation and for evaluating your own custom or imported models after fine-tuning. Think of it as the AWS Console's built-in prompt IDE for Bedrock. No code required.

4. FINE-TUNING IN BEDROCK

Creating Custom Models

Fine-tuning lets you **adapt a pre-existing foundation model to your specific use case** by running additional supervised training on your own labelled data. The result is a "Custom Model" that can be used exactly like any base model.

Why Fine-Tune?

Problem	Without Fine-Tuning	With Fine-Tuning
Custom tone/style	Requires long system prompts every time	Baked into the model weights
Domain knowledge	Model may hallucinate or be vague	Model trained on your domain data
Recent proprietary data	Not available to base model	Included in training
Token efficiency	Must repeat context in every prompt (costly)	Saves tokens — context is in weights
Specific tasks	May need prompt engineering tricks	Model directly learns the task

How Fine-Tuning Works in Bedrock

Supported Models:

- **Titan** text models (Amazon)
- **Cohere** Command models
- **Meta Llama** models

Training Data Format (Text Models):

Provide **labelled pairs** of prompts and completions as newline-delimited JSON (JSONL), uploaded to an S3 bucket:

```
{"prompt": "What is your return policy?", "completion": "You may return items within 30 days..."}
{"prompt": "How do I track my order?", "completion": "Log in and visit My Orders..."}
```

Training Data Format (Image Models):

- Provide pairs of S3 image paths and their text descriptions (prompts).
- Used for text-to-image models (Stable Diffusion) or image-to-embedding models.

Step	Detail
1. Prepare data	JSONL file with prompt/completion pairs; images + descriptions for image models
2. Upload to S3	Place training data in an S3 bucket accessible to Bedrock
3. Create job	Console or API: CreateModelCustomizationJob; select base model, S3 path, hyperparams

4. Training runs	AWS manages the compute — this can be expensive, especially for large models
5. Custom model ready	Resulting model appears as a custom model; invoke it like any base model
6. Iterate	You can fine-tune a fine-tuned model again to make it "smarter" over time

■ Exam Tip — Fine-Tuning Data Must Be in S3

Training data for Bedrock fine-tuning is **always sourced from Amazon S3**. You cannot pass training data inline. For sensitive training data, use a **VPC with PrivateLink** so data never traverses the public internet.

■ Key Note — Applications of Fine-Tuning

- Customer support chatbot with a specific personality.
- Sales assistant trained on past email threads.
- Classification model trained on your labelled business data.
- Model updated with recent events not in the original training data.

5. CONTINUED PRE-TRAINING

Fine-Tuning with Unlabelled Data

Continued Pre-Training (CPT) is similar to fine-tuning, but instead of labelled prompt–completion pairs, you provide **raw unlabelled text**. The model learns to predict the next token from this text, absorbing domain familiarity without being taught specific Q&A; pairs.

Aspect	Fine-Tuning (Custom Models)	Continued Pre-Training
Data type	Labelled: {"prompt": "...", "completion": "..."}	Unlabelled: {"input": "plain text..."}
Purpose	Teach specific tasks, styles, or Q&A; behaviour	Familiarise model with domain terminology and concepts
Use case	Customer support bot, classification, ad writing	Legal documents, medical literature, internal wikis
Data volume	Can work with hundreds to thousands of examples	Often needs larger corpus of text (full documents)
Model learns	Input → output mappings (supervised)	Language patterns in the domain (unsupervised)

Continued Pre-Training Data Format:

```
{"input": "Amortisation is the process of spreading loan payments over time."}
{"input": "A derivative is a financial contract whose value depends on an underlying asset."}
{"input": "The Federal Reserve sets the benchmark interest rate for US banks."}
```

■ Key Note — When to Choose CPT vs Fine-Tuning

Use Continued Pre-Training when you want the model to become fluent in your domain vocabulary and concepts without teaching it specific outputs — e.g., exposing it to your entire internal knowledge base. **Use Fine-Tuning** when you want the model to produce specific types of responses, follow a format, or take on a specific role or persona.

■ Pro Tip — Data Goes Into the Model

Both CPT and fine-tuning effectively embed additional knowledge into the model weights themselves. This means you don't need to include that knowledge in your prompts every time — which saves tokens and reduces latency. This is different from RAG, where knowledge stays in an external database and is retrieved at query time.

6. RETRIEVAL AUGMENTED GENERATION (RAG)

Open-Book Exams for LLMs

RAG is a technique for giving an LLM access to **up-to-date or proprietary information at query time** without baking it into the model weights. Think of it as giving the LLM an "open-book exam" — you look up the answer in a reference database, then hand the relevant text to the LLM as context for its response.

How RAG Works — Step by Step

Step	What Happens
1. User asks a question	"What did the President say in his speech yesterday?"
2. Query encoding	The question is converted into an embedding vector.
3. Vector search (Retrieval)	The embedding is compared against a vector database; the top-N most similar document chunks are returned.
4. Context injection (Augmentation)	The retrieved chunks are inserted into the prompt: "Answer this question using the following article: [chunk]..."
5. LLM generates response	The LLM reads the augmented prompt and produces a grounded answer.

RAG: Pros and Cons

■ Advantages

- Faster and cheaper than fine-tuning for incorporating new data.
- Updating information is just a matter of updating the database — no retraining.
- Enables semantic search via vector similarity.
- Helps prevent hallucinations on unknown topics by grounding answers.
- You are not "training" the model — no GPU compute required.
- Great for "AI search" use cases your organisation wants quickly.

■ Limitations

- Effectively a very sophisticated search engine — can feel over-engineered.
- Very sensitive to prompt templates used to inject retrieved context.
- Non-deterministic — same question may retrieve different chunks.
- Can still hallucinate if retrieved context is irrelevant or wrong.
- Quality is highly dependent on the relevancy of retrieved documents.
- Chunking strategy has a big impact on retrieval quality.

■ Exam Tip — RAG vs Fine-Tuning

This is one of the most tested trade-offs in the exam. **RAG**: use when data changes frequently, when you need to cite sources, or when you cannot afford to retrain. **Fine-tuning**: use when you need consistent style/format, when data is stable, or when latency and token cost of prompts matter. They are **complementary** — a fine-tuned model can still use RAG.

RAG Architecture — The Flow

Component	Role in RAG	AWS Service Options
Query Encoder	Converts user question into embedding vector	Amazon Titan Embeddings, Cohere Embed
Vector Store	Stores document embeddings; retrieves top-N similar chunks	OpenSearch Serverless, Aurora, MemoryDB, Pinecone, Redis
Retriever	Orchestrates search and returns relevant document chunks	Bedrock Knowledge Bases (automatic), custom code
Generator (LLM)	Reads augmented prompt (context + question) and generates response	Any Bedrock FM (Claude, Titan, Llama, etc.)

7. EMBEDDINGS & VECTOR DATABASES

The Engine Behind Semantic Search

What Are Embeddings?

An **embedding** is a high-dimensional numerical vector (typically hundreds to thousands of numbers) that represents the **semantic meaning** of a piece of text, image, or other data. The key property: **similar items have similar (close) embeddings** in vector space, while unrelated items are far apart.

- Example: "The Starship Enterprise" and "Spaceballs" will have embeddings close to each other (both are sci-fi), but far from "potato" or "rhubarb" (vegetables).
- Embeddings are computed by **embedding models** — in Bedrock, use Amazon Titan Embeddings or Cohere Embed.
- You must have model access for the embedding model before creating a Knowledge Base.

Vector Databases

A vector database stores your data **alongside its precomputed embedding vectors**. When you search, you compute an embedding for your query and find the stored embeddings that are closest — this is called **K-Nearest Neighbor (KNN) search** or "vector search".

Category	Options	Notes
AWS-native (vector-capable)	Amazon OpenSearch Serverless, OpenSearch Service, Amazon Aurora (pgvector), Amazon MemoryDB	OpenSearch Serverless is the default for Bedrock Knowledge Bases in dev
Managed third-party	Pinecone, Redis Enterprise Cloud, MongoDB Atlas	Supported as vector stores in Bedrock Knowledge Bases
Open-source (self-managed)	Chroma, Milvus, Qdrant, Weaviate, LanceDB, Vespa, Marqo	You manage hosting; not directly integrated with Bedrock
Traditional DBs (vector-extended)	Elasticsearch/OpenSearch (text + vector), Redis, Neptune, Cassandra	Can do vector search alongside traditional queries

How Vector Retrieval Works

- **1. Ingest:** Each document chunk is passed through an embedding model to produce a vector.
- **2. Store:** The vector + original chunk text is stored in the vector database.
- **3. Query:** At search time, the user's question is passed through the same embedding model.
- **4. Search:** The vector DB finds the top-N stored vectors closest to the query vector (KNN).
- **5. Return:** The original text chunks corresponding to those vectors are returned as context.

■ Key Note — Chunking Strategy

When loading documents into a vector store, you must decide how to **chunk** them — how many tokens each vector represents. **Too small:** each chunk lacks context, retrieval may miss meaning. **Too large:** chunks contain irrelevant content that dilutes the embedding. Bedrock Knowledge Bases lets you configure chunking strategy and token size.

8. BEDROCK KNOWLEDGE BASES

Managed RAG — Chat With Your Data

Bedrock Knowledge Bases is AWS's **fully managed RAG implementation**. You point it at your documents, it automatically chunks, embeds, and stores them in a vector store — and then you query it via API or connect it to an agent.

Data Sources Supported

Data Sources

- **Amazon S3** — upload PDFs, text, HTML, markdown, JSON (with schema)
- **Web Crawler** — crawl publicly accessible web pages
- **Confluence** — connect to Confluence workspaces
- **Salesforce** — connect to Salesforce records and documents
- **SharePoint** — connect to SharePoint document libraries

Configuration Options

- **Embedding model required:** currently Cohere Embed or Amazon Titan Embeddings.
- You **must request model access** for the embedding model first.
- You can control the **vector dimension** (affects accuracy vs storage).
- **Chunking control:** choose default, fixed-size, semantic, hierarchical, or custom chunking.

Supported Vector Stores

Vector Store	Notes
Amazon OpenSearch Serverless	Default for development; fully serverless; easy setup
Amazon Aurora (PostgreSQL with pgvector)	Relational + vector in one place
Amazon MemoryDB	In-memory; now supports vector capabilities; ultra-low latency
MongoDB Atlas	Popular managed document + vector DB
Pinecone	Purpose-built vector DB; enterprise option
Redis Enterprise Cloud	In-memory vector search; very fast

How to Use Knowledge Bases

"Chat with your document"

Ask natural language questions about uploaded PDFs, wikis, or other docs. Bedrock retrieves relevant chunks and generates a grounded answer.

Integrate into your application directly

Call `RetrieveAndGenerate` via the `bedrock-agent-runtime` API to add RAG to any custom application.

Incorporate into an agent ("Agentic RAG")

Connect a knowledge base to a Bedrock Agent. The agent decides when to use the KB vs. call other tools. RAG becomes just another tool the agent can invoke.

■ **Exam Tip — Knowledge Bases vs Fine-Tuning vs CPT**

Knowledge Base (RAG): data stays external; easy to update; used at query time. **Fine-Tuning:** labelled data baked into weights; for task/style adaptation. **CPT:** unlabelled data baked into weights; for domain vocabulary. Exam may present scenarios like "data changes weekly" → RAG wins. "Needs consistent format" → Fine-tuning wins.

9. AMAZON BEDROCK GUARDRAILS

Content Safety for GenAI Applications

Bedrock Guardrails is a **content safety and compliance layer** that sits between your application and the LLM. It filters both incoming prompts and outgoing responses, preventing harmful, irrelevant, or sensitive content from reaching users.

Guardrail Filter Types

Filter Type	What It Does	Example Use
Word Filtering	Block or redact specific words or phrases from prompts and responses.	Block competitor brand names from a customer service bot.
Topic Filtering	Prevent the model from discussing defined topics entirely.	Block discussions of politics on a cooking assistant.
Profanity Filtering	Automatically detect and block profane or offensive language.	Family-friendly application or workplace tool.
PII Detection & Removal	Detect personally identifiable information (names, SSNs, credit cards) and remove or mask it.	Customer support bot that shouldn't store PII in logs.
Contextual Grounding Check	Evaluates whether the response is grounded in retrieved context and relevant to the query. Helps prevent hallucinations.	RAG applications where factual accuracy is critical.

Contextual Grounding Check — Explained

The Contextual Grounding Check is particularly important for RAG applications. It measures two dimensions:

- **Grounding score:** How closely does the response align with the retrieved context? A low score means the model is "making things up" rather than using the provided data.
- **Relevance score:** Does the response actually address what the user asked? A low score means the model went off-topic.
- You set **threshold values** for both; responses below the threshold are blocked and replaced with your configured "blocked message".

Integration Points

- Guardrails can be applied directly to **text foundation model invocations** (Converse, InvokeModel).
- Can be integrated into **Bedrock Agents** to filter agent prompts and responses.
- Can be integrated into **Knowledge Bases** to filter RAG-generated answers.
- You configure a custom **"blocked message"** — what users see when content is filtered.

■ Exam Tip — Guardrails ≠ Model Safety

Guardrails add a layer on top of whatever safety is already built into the model. They are **configurable by you for your business rules** — not a replacement for inherent model safety. Key things tested: Guardrails filter **both prompts and responses**; they support **PII masking**; the Contextual Grounding Check helps with **hallucination prevention**.

10. LLM AGENTS IN BEDROCK

Giving Tools to Your Language Model

An **LLM Agent** is a language model that has been given the ability to **call external tools** (functions, APIs, databases) to help answer a request. Rather than just generating text, an agent can plan a multi-step approach, call tools to gather information or perform actions, and synthesise results into a final response.

Conceptual Architecture of an Agent

Component	What It Really Is	In Bedrock
Agent Core (LLM)	The foundation model that reasons and decides what to do.	Any supported Bedrock FM (Claude, Llama, Titan, etc.)
Memory	The chat history + external data stores (not true persistent memory).	Session context + Knowledge Base
Planning Module	Guidance given to the LLM on how to decompose a question into sub-tasks.	System prompt / "agent instructions" you write
Tools Core	External functions the LLM can call to take actions or get information.	Lambda functions (via Action Groups)

Action Groups — How Tools Are Defined

In Bedrock, each tool is defined as an **Action Group**. An Action Group wires a natural-language description to a **Lambda function** that executes when the agent decides the tool is needed.

- **Agent prompt:** High-level system prompt telling the agent what it is and what it can do.
- **Action Group name:** e.g., "GetWeather"
- **Description: Very important** — the LLM reads this to decide when to call the tool. E.g., "Use this function to determine the current weather in a city." The description must be precise.
- **Parameters:** Define each parameter by name, type, description, and whether it is required. The LLM extracts these from the user's message.
- **Lambda function:** The actual code that runs when the agent calls the tool. Can call any external API, database, or service.
- **Schema:** Defined using OpenAPI-style JSON or via the Bedrock UI table.
- **Ask for missing info:** You can configure the agent to ask the user for parameters it cannot extract from the prompt.

Agentic RAG

Agents can have **Knowledge Bases attached to them**. When a question is best answered by retrieving from the knowledge base, the agent uses it as a tool. A prompt tells the LLM when to consult the KB: "Use this for answering questions about our product documentation."

- This is called **Agentic RAG** — RAG is just another tool in the agent's toolkit.
- **Code Interpreter:** An optional capability allowing the agent to write and execute code to answer analytical questions or produce charts.

Agent Execution Flow

- **User sends request:** e.g., "What's the weather in New York and how does it compare to our sales data?"

- **Agent decomposes:** Plans: (1) call GetWeather tool for NYC, (2) query knowledge base for sales data.
- **Calls Action Group:** Invokes Lambda function for weather API; Retrieve from KB for sales.
- **Receives results:** Lambda returns weather JSON; KB returns relevant sales chunk.
- **Augments prompt:** Builds a new prompt with both results as context.
- **Generates response:** FM synthesises a final answer combining weather + sales data.

■ **Exam Tip — Action Groups = Lambda + Description**

The **description of each Action Group parameter** is critically important — the LLM uses it to extract the value from the user's message. A poorly worded description = the agent fails to extract parameters correctly. Also remember: tools in Bedrock Agents are always backed by **Lambda functions**.

11. DEPLOYING BEDROCK AGENTS

Aliases, Throughput & Invocation

Once an agent is built and tested, you deploy it by creating an **alias**. An alias is a pointer to a specific version of your agent — it creates a **deployed, immutable snapshot** that your application can invoke reliably.

Concept	Description
Agent Alias	A named reference to a specific agent version. Create a new alias to deploy an updated agent without changing the endpoint your app calls.
On-Demand Throughput (ODT)	The agent runs within the account-level service quotas. No upfront cost; pay per request. Fine for most workloads.
Provisioned Throughput (PT)	Purchase a committed rate (tokens per minute) and request count. Guarantees capacity for high-volume production agents.
InvokeAgent API	Your application calls InvokeAgent with the agent alias ID. Bedrock handles orchestration, tool calling, and response generation.
Endpoint	Use the Agents for Amazon Bedrock Runtime Endpoint in the bedrock-agent-runtime namespace.

■ Exam Tip — ODT vs PT

On-Demand Throughput: no commitment, pay per use, subject to account quotas. **Provisioned Throughput:** reserved capacity, higher cost, guaranteed performance. Use PT when you need predictable latency and have high or bursty traffic. Use ODT for development, testing, or moderate workloads.

■ Key Note — Version Control with Aliases

Aliases enable **blue/green-style deployments** for agents. You can point your application at an alias, update the agent, create a new alias for the updated version, and switch traffic over without downtime. The old alias keeps pointing to the old version until you update it.

12. MORE BEDROCK FEATURES

Imported Models, Evaluation, Studio & More

Imported Models

You can import custom models you have trained yourself (outside of Bedrock) directly into Bedrock from **Amazon S3** or from **Amazon SageMaker**. Once imported, the model can be used via the same Bedrock APIs as any other model.

Model Evaluation

Bedrock includes a built-in model evaluation framework to assess quality before deploying. Two modes:

Evaluation Type	Method	Details
Automatic	Algorithm-based scoring	Accuracy, toxicity, robustness, BERTScore, F1, exact match. Test against your own prompts or AWS-provided benchmark datasets.
Human (your team)	Manual review	Bring your own team of reviewers to evaluate model responses.
Human (AWS-managed)	AWS provides reviewers	AWS manages a team of human reviewers on your behalf.

Provisioned Throughput

Available for base models as well as agents. Reserve compute capacity for predictable, high-volume inference. Billed per hour regardless of usage.

Watermark Detection

Bedrock can detect whether an image was generated by an Amazon Titan image model. This supports responsible AI and content authenticity use cases.

Bedrock Studio

A web-based workspace for Bedrock that does not require AWS account credentials. Integrates with **AWS IAM Identity Center (SSO)** and your own Identity Provider (IdP). Teams can collaborate on Bedrock projects and components without each needing AWS access.

■ Exam Tip — Bedrock Studio

Bedrock Studio is for **teams that need to use Bedrock without individual AWS accounts**. It uses SSO (IAM Identity Center) with your existing Identity Provider. Distinguisher: if a scenario mentions "non-technical users collaborating on GenAI projects" or "no AWS account required", Bedrock Studio is the answer.

■ Key Note — Watermark Detection

Titan image models embed an **invisible watermark** in generated images. Bedrock's watermark detection API can verify whether an image has this watermark, helping identify AI-generated content. This is part of AWS's Responsible AI approach.

MASTER CHEAT SHEET — SECTION 8

Amazon Bedrock Quick Reference

API Endpoints

bedrock	Control plane: manage/deploy/train models
bedrock-runtime	Inference: Converse, InvokeModel, InvokeModelWithResponseStream
bedrock-agent	Control plane: manage agents + knowledge bases
bedrock-agent-runtime	Inference: InvokeAgent, Retrieve, RetrieveAndGenerate

Model Customisation Techniques

Fine-Tuning (Custom Models)	Labelled prompt→completion JSONL in S3. Titan, Cohere, Meta models. Teaches specific tasks, style, persona. Expensive.
Continued Pre-Training	Unlabelled {"input": "text"} JSONL. Familiarises model with domain. No labels needed. Good for large document corpora.
RAG (Knowledge Bases)	External vector DB queried at runtime. Easy to update. No training cost. Still can hallucinate if retrieval quality is poor.

Guardrail Filter Types

Word Filter	Block specific words/phrases
Topic Filter	Block entire discussion topics
Profanity Filter	Block offensive language
PII Removal/Masking	Detect + remove/mask personally identifiable information
Contextual Grounding	Score grounding (vs context) + relevance (vs query); block if below threshold

Agents Quick Reference

Tools = Action Groups	Each Action Group → Lambda function + description + parameters
Description is critical	LLM reads it to know when + how to call the tool
Agentic RAG	KB attached to agent = RAG becomes just another tool
Code Interpreter	Optional: agent writes + runs code to answer analytical questions
Deploy via Alias	Alias = versioned snapshot; InvokeAgent with alias ID
ODT vs PT	On-Demand: account quota limits; Provisioned: reserved capacity

■ Top Exam Traps for Bedrock

1. **Bedrock is always serverless** — never requires instance provisioning. 2. **Model access must be requested separately** from IAM permissions. 3. **Root account cannot use Bedrock** — must use IAM user/role. 4. **Fine-tuning data must be in S3**. 5. **RAG = update database; Fine-tuning = retrain model**. 6. **Action Group description is what the LLM reads to decide when to call the tool**. 7. **Watermark detection** = Titan image models only. 8. **Bedrock Studio** = no AWS account needed, uses SSO/IdP.

■ Vector DB Options for Knowledge Bases

Default (dev): OpenSearch Serverless. **Also supported:** Aurora (pgvector), MemoryDB, MongoDB Atlas, Pinecone, Redis Enterprise Cloud. **Embedding models:** Amazon Titan Embeddings OR Cohere Embed (must request access first).